

Problem 2: Option 2: Memcached.....	3
Instructions.....	3
Code and General Explanation.....	5
Initialization.....	7
Graphs.....	7
General graphs.....	7
1.....	7
2.....	8
3.....	9
4.....	10
Evictions.....	11
1.....	11
2.....	12
3.....	12
4.....	12
Memory.....	13
1.....	13
2.....	13
3.....	14
4.....	14
Final print out in the terminal.....	15
1.....	15
2.....	16
3.....	17
4.....	18
Explanation of the graphs.....	21
Notable Relationships.....	22
Notes on graph 4 and shape parameters.....	23
Reflection.....	23

Problem 2: Option 2: Memcached

Instructions

Downloaded package for memcached: <https://pecl.php.net/package/memcache>

then run ``brew install memcached`` in the terminal.

install requirements.txt

This is for the inject.py file which injects the nodes/ports and then makes a graph based on the output. More details below.

Here is a good reference from the documentation for configuring the initial state:

<https://docs.memcached.org/serverguide/configuring/>

To create our initial engine we must create 5 'peers' All we do for this is create 5 different TCP ports for each 'peer.' As in most of the examples we will use -m 64, set the -d and just use ports starting from 11211 (default) and going up.

```
memcached -p 11211 -m 64 -d
```

```
memcached -p 11212 -m 64 -d
```

```
memcached -p 11213 -m 64 -d
```

```
memcached -p 11214 -m 64 -d
```

```
memcached -p 11215 -m 64 -d
```

if we want to check evictions we can change this to

```
memcached -p 11211 -m 2 -d
```

```
memcached -p 11212 -m 2 -d
```

```
memcached -p 11213 -m 2 -d
```

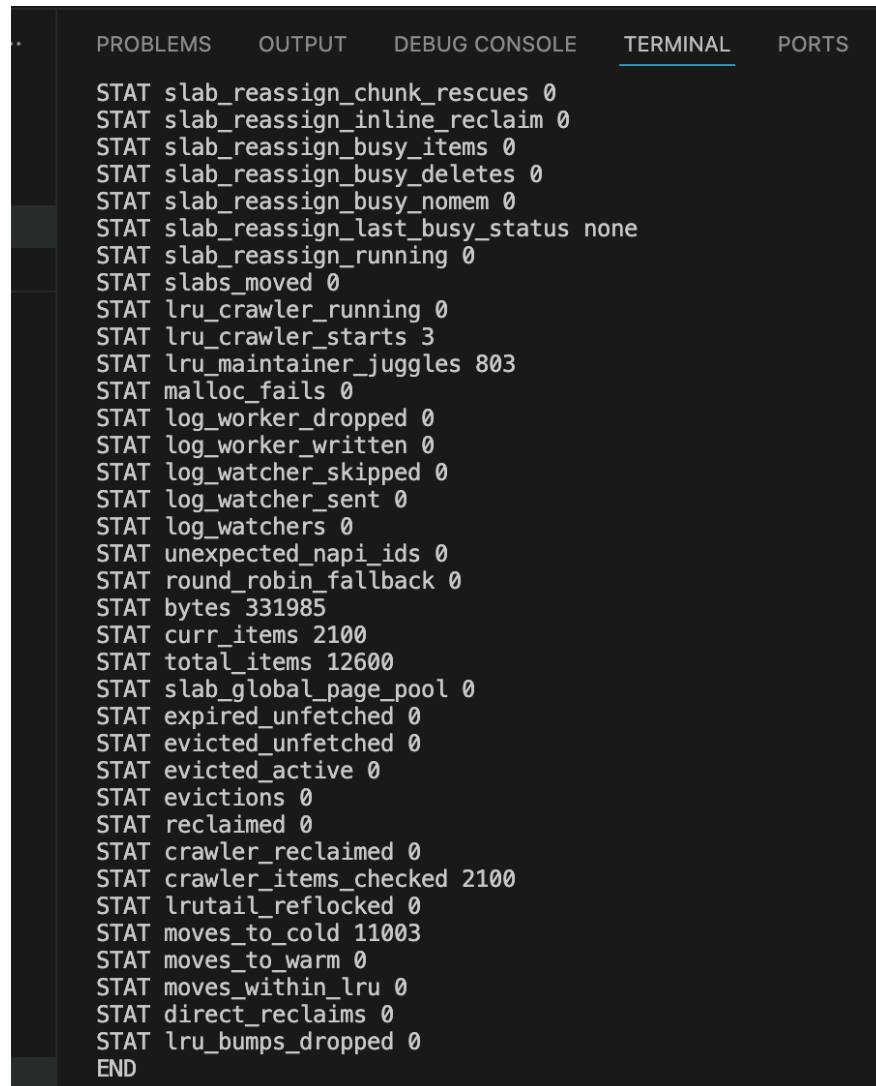
```
memcached -p 11214 -m 2 -d
```

```
memcached -p 11215 -m 2 -d
```

We can check they are running by looking for them on all running process and we can stop all of them using `killall memcached`

An example of the stats we can grab using this cmd: `echo stats | nc 127.0.0.1`

{p} where {p} is a specific port



The screenshot shows a terminal window with a dark background. At the top, there are five tabs: 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected and underlined), and 'PORTS'. The terminal displays the output of the 'echo stats | nc 127.0.0.1' command, showing various memcached statistics. The output is as follows:

```
STAT slab_reassign_chunk_rescues 0
STAT slab_reassign_inline_reclaim 0
STAT slab_reassign_busy_items 0
STAT slab_reassign_busy_deletes 0
STAT slab_reassign_busy_nomem 0
STAT slab_reassign_last_busy_status none
STAT slab_reassign_running 0
STAT slabs_moved 0
STAT lru_crawler_running 0
STAT lru_crawler_starts 3
STAT lru_maintainer_juggles 803
STAT malloc_fails 0
STAT log_worker_dropped 0
STAT log_worker_written 0
STAT log_watcher_skipped 0
STAT log_watcher_sent 0
STAT log_watchers 0
STAT unexpected_napi_ids 0
STAT round_robin_fallback 0
STAT bytes 331985
STAT curr_items 2100
STAT total_items 12600
STAT slab_global_page_pool 0
STAT expired_unfetched 0
STAT evicted_unfetched 0
STAT evicted_active 0
STAT evictions 0
STAT reclaimed 0
STAT crawler_reclaimed 0
STAT crawler_items_checked 2100
STAT lru_tail_reflocked 0
STAT moves_to_cold 11003
STAT moves_to_warm 0
STAT moves_within_lru 0
STAT direct_reclaims 0
STAT lru_bumps_dropped 0
END
```

```

error on 11215: invalid literal for int() with base 10: '1.6.42'
STAT pid 85668
STAT uptime 271
STAT time 1780609591
STAT version 1.6.42
STAT libevent 2.1.12-stable
STAT pointer_size 64
STAT rusage_user 0.073842
STAT rusage_system 0.135883
STAT max_connections 1024
STAT curr_connections 3
STAT total_connections 9
STAT rejected_connections 0
STAT connection_structures 4
STAT response_obj_oom 0
STAT response_obj_count 1
STAT response_obj_bytes 65536
STAT read_buf_count 8
STAT read_buf_bytes 131072
STAT read_buf_bytes_free 49152
STAT read_buf_oom 0
STAT reserved_fds 20
STAT cmd_get 12294
STAT cmd_set 12600
STAT cmd_flush 0
STAT cmd_touch 0
STAT cmd_meta 0
STAT get_hits 10044
STAT get_misses 2250
STAT get_expired 0
STAT get_flushed 0
STAT delete_misses 0
STAT delete_hits 0
STAT incr_misses 0
STAT incr_hits 0
STAT decr_misses 0
STAT decr_hits 0
STAT cas_misses 0
STAT cas_hits 0
STAT cas_badval 0
STAT touch_hits 0
STAT touch_misses 0
STAT store_too_large 0
STAT store_no_memory 0
STAT auth_cmds 0
STAT auth_errors 0
STAT bytes_read 1668011
STAT bytes_written 1207241
STAT limit_maxbytes 67108864
STAT accepting_conns 1
STAT listen_disabled_num 0
STAT time_in_listen_disabled_us 0
STAT threads 4
STAT conn_yields 0
STAT hash_power_level 16
STAT hash_bytes 524288
STAT hash_is_expanding 0
STAT slab_reassign_rescues 0

```

Code and General Explanation

To begin this project there are a few steps.

1. Set up the nodes (as seen in instructions)
2. Inject data

3. get the stats from memcached and make graphs and analysis

My initial idea was to just inject specific data but all the ports looked the same and that wouldn't give any real data if I manually set the hit and miss ratio. Instead I used a zipfilan distribution, which is used to mimic real behavior in web servers. This improved the quality of my data simulation.

Furthermore I grabbed a lot of data from stats and made a multitude of graphs to show how memcache works and its performance. I ran this a couple of times. Since upon a normal run its difficult to fill up all 64 mb of memory I ran the program on one with 64 and one with 2 to see the difference in eviction data. Inside inject.py there are also constants that can be changed like the total users and also the shape parameter of the distribution. The shape parameter defines how 'steep' the data is, that is the difference in weight between the first and second and so on. That is, with a high shape parameter the results will vary more by port.

The runs were as follows:

1. TOTAL_USERS: 50,000

SHAPE_PARAM: 1.2

(ran with -m 64)

2. TOTAL_USERS: 50,000

SHAPE_PARAM: 1.2

(ran with -m 2)

3. TOTAL_USERS: 100,000

SHAPE_PARAM: 1.2

(ran with -m 2)

4. TOTAL_USERS: 50,000

SHAPE_PARAM: 2

(ran with -m 2)

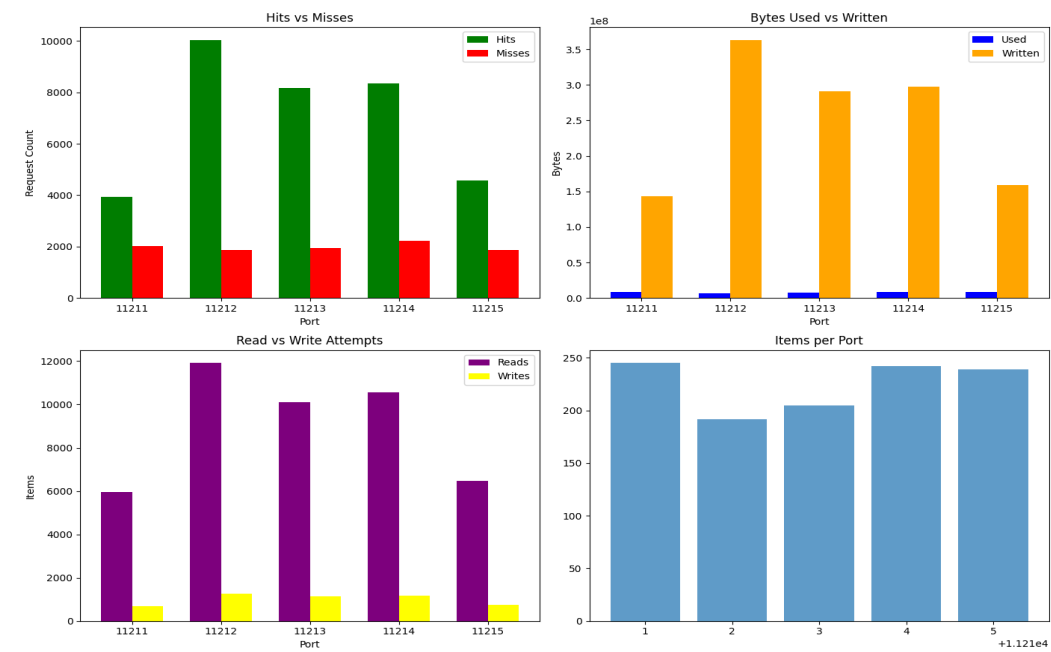
Initialization

```
isabelledarling@Isabelles-MacBook-Air memcache-8.2 % memcached -p 11211 -m 2 -d
memcached -p 11212 -m 2 -d
memcached -p 11213 -m 2 -d
memcached -p 11214 -m 2 -d
memcached -p 11215 -m 2 -d
isabelledarling@Isabelles-MacBook-Air memcache-8.2 % ps aux | grep memcached
isabelledarling  94455  0.0  0.0  410199728  1184 s002  S+   7:21PM  0:00.00 grep memcached
isabelledarling  94362  0.0  0.0  410494960  3072  ??   Ss    7:21PM  0:00.00 memcached -p 11215 -m 2 -d
isabelledarling  94360  0.0  0.0  410485744  3024  ??   Ss    7:21PM  0:00.00 memcached -p 11214 -m 2 -d
isabelledarling  94358  0.0  0.0  410483696  2976  ??   Ss    7:21PM  0:00.00 memcached -p 11213 -m 2 -d
isabelledarling  94356  0.0  0.0  410493936  3040  ??   Ss    7:21PM  0:00.00 memcached -p 11212 -m 2 -d
isabelledarling  94354  0.0  0.0  410484720  2992  ??   Ss    7:21PM  0:00.00 memcached -p 11211 -m 2 -d
isabelledarling@Isabelles-MacBook-Air memcache-8.2 %
memcached -p 11211 -m 64 -d
memcached -p 11212 -m 64 -d
memcached -p 11213 -m 64 -d
memcached -p 11214 -m 64 -d
memcached -p 11215 -m 64 -d
isabelledarling@Isabelles-MacBook-Air memcache-8.2 % ps aux | grep memcached
isabelledarling  93536  0.0  0.0  410063264   128 s002  R+    7:19PM  0:00.00 grep memcached
isabelledarling  93449  0.0  0.0  410756080  3104  ??   Ss    7:19PM  0:00.00 memcached -p 11215 -m 64 -d
isabelledarling  93447  0.0  0.0  410484720  3008  ??   Ss    7:19PM  0:00.00 memcached -p 11214 -m 64 -d
isabelledarling  93445  0.0  0.0  410484720  3008  ??   Ss    7:19PM  0:00.00 memcached -p 11213 -m 64 -d
isabelledarling  93443  0.0  0.0  410492912  3024  ??   Ss    7:19PM  0:00.00 memcached -p 11212 -m 64 -d
isabelledarling  93441  0.0  0.0  410502128  3056  ??   Ss    7:19PM  0:00.00 memcached -p 11211 -m 64 -d
isabelledarling@Isabelles-MacBook-Air memcache-8.2 %
```

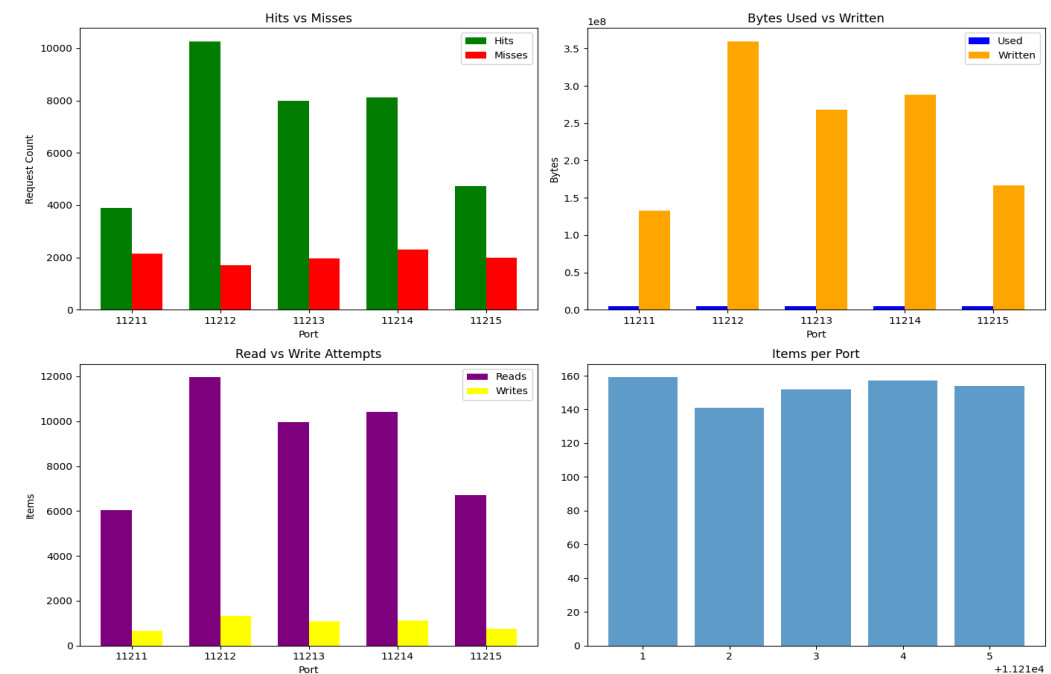
Graphs

General graphs

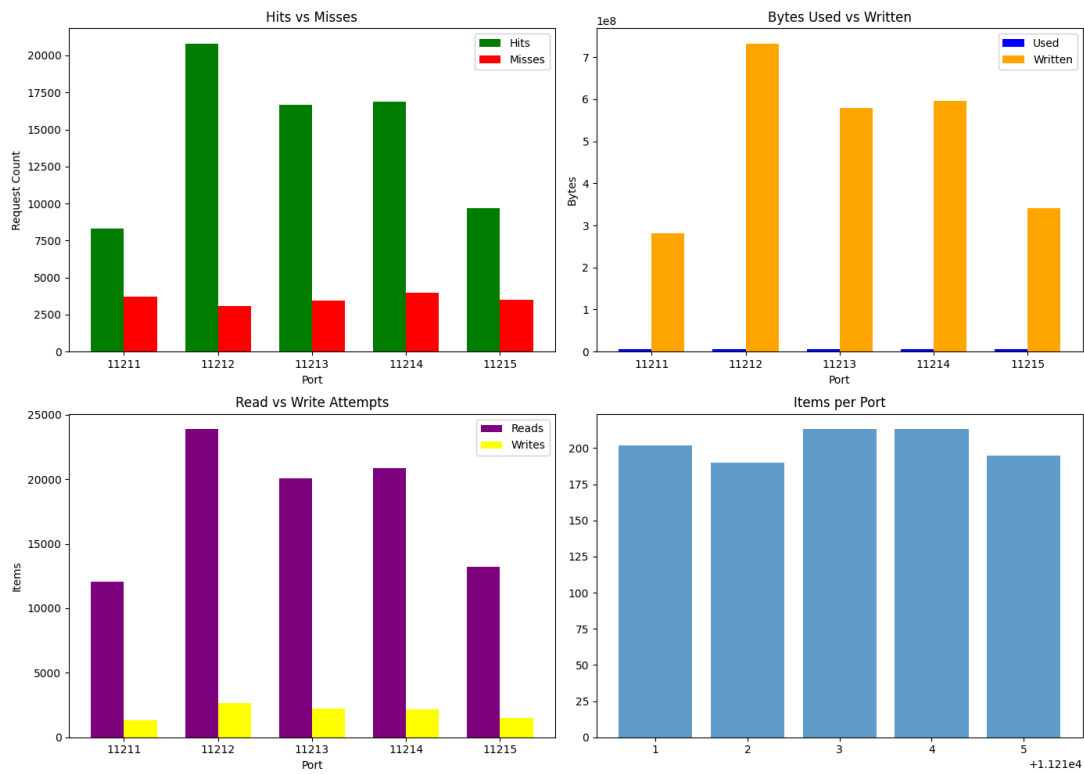
1



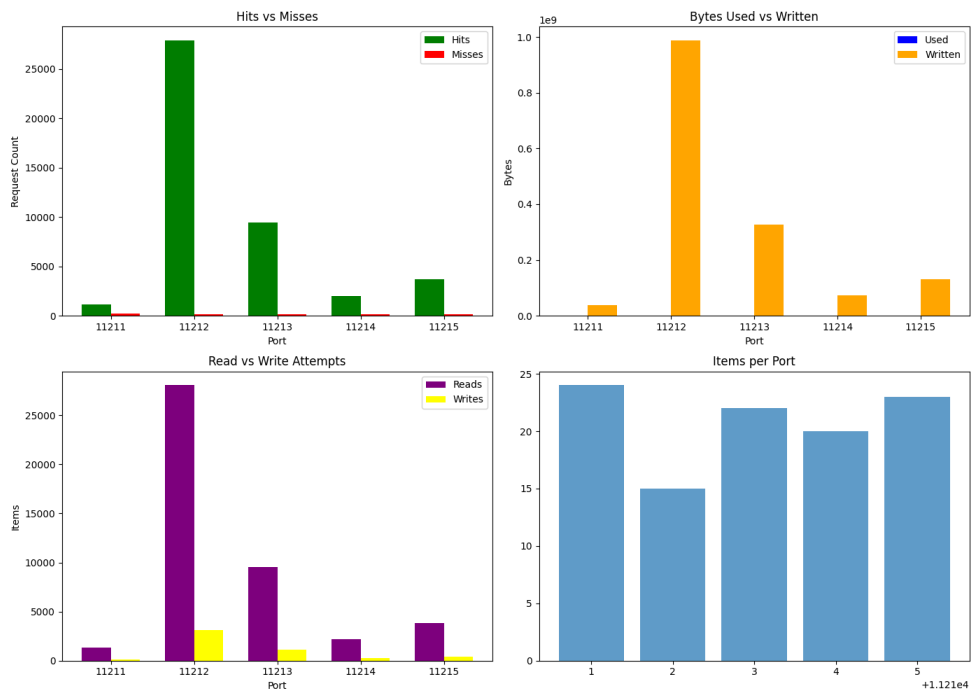
2



3

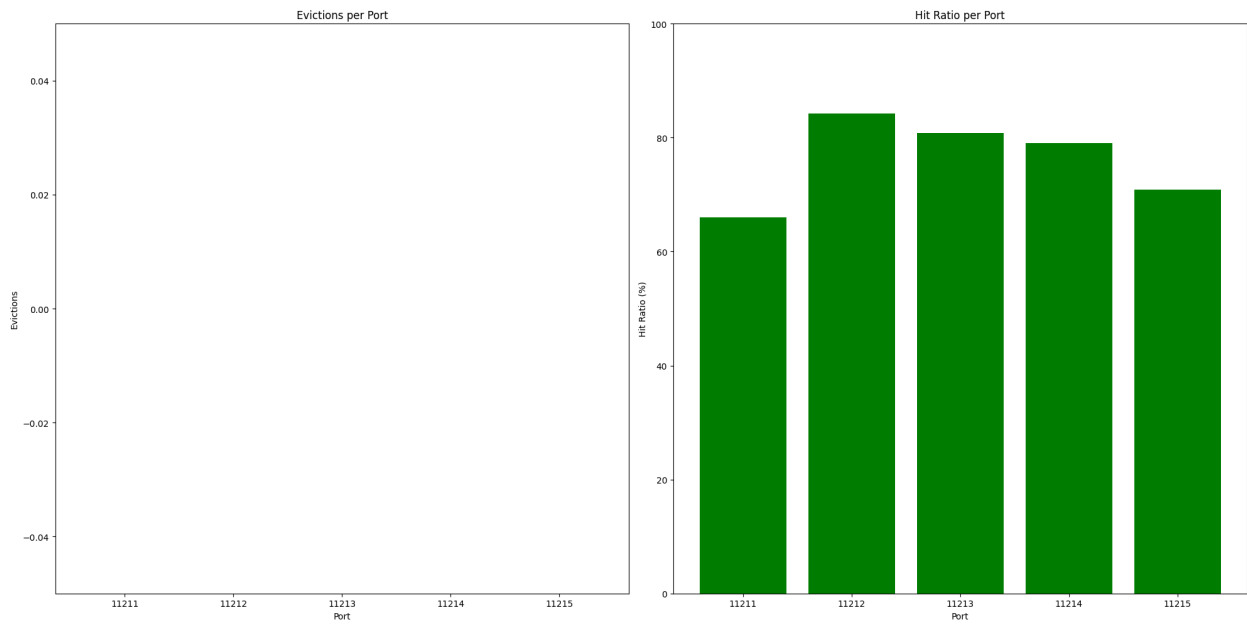


4

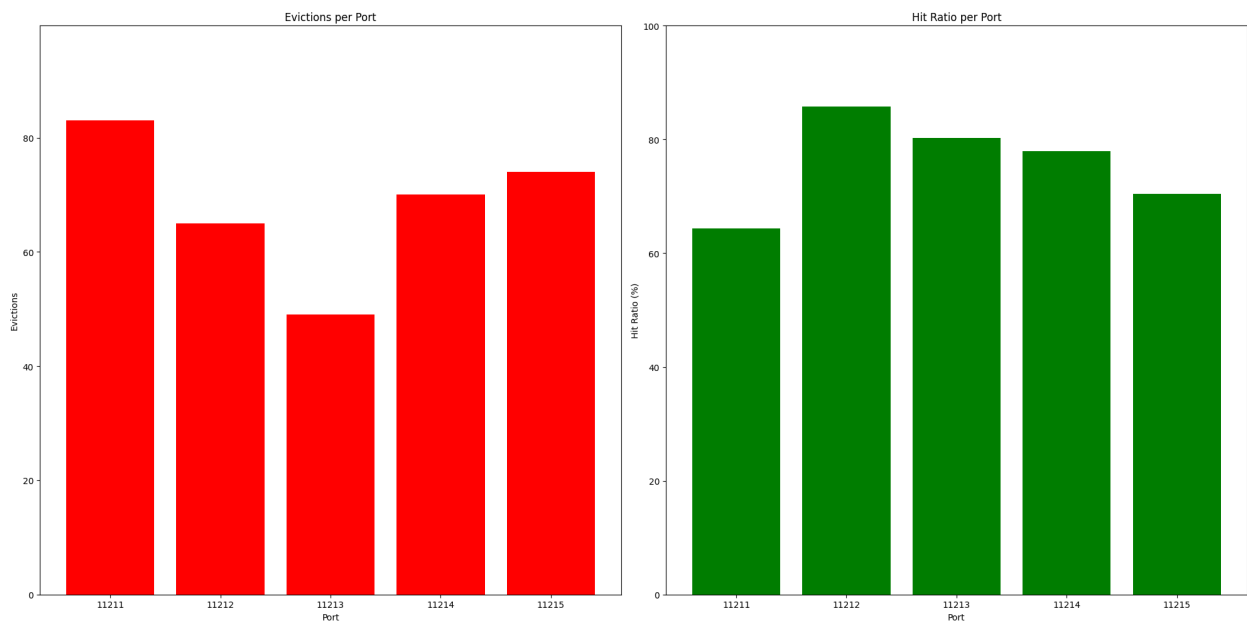


Evictions

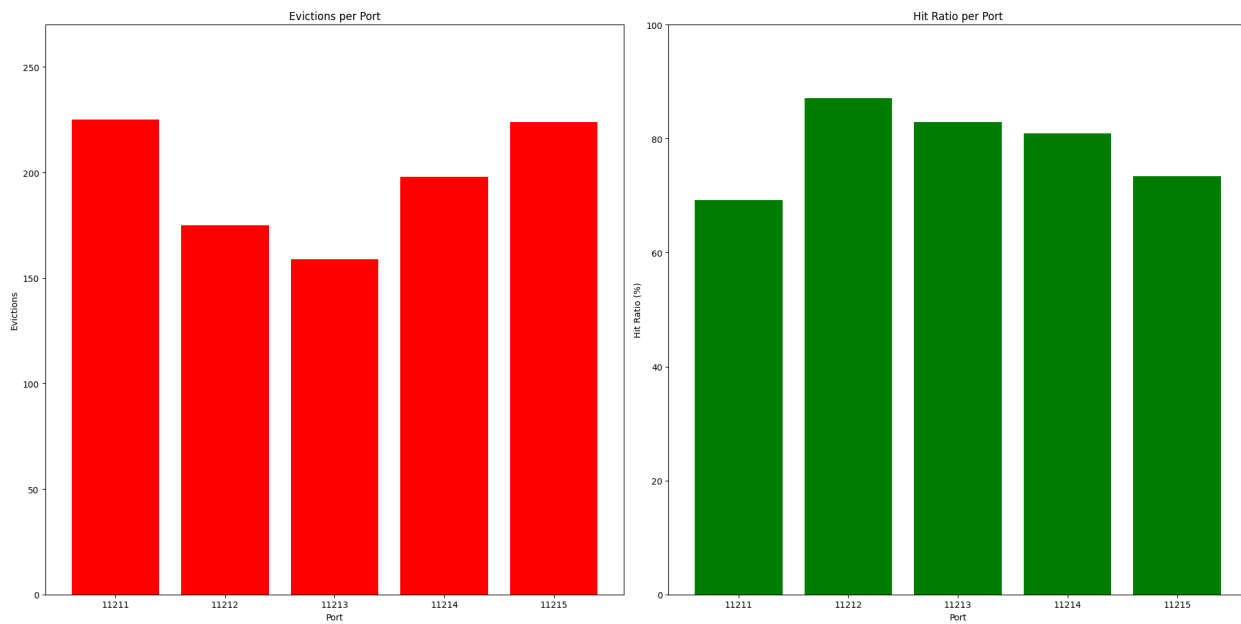
1



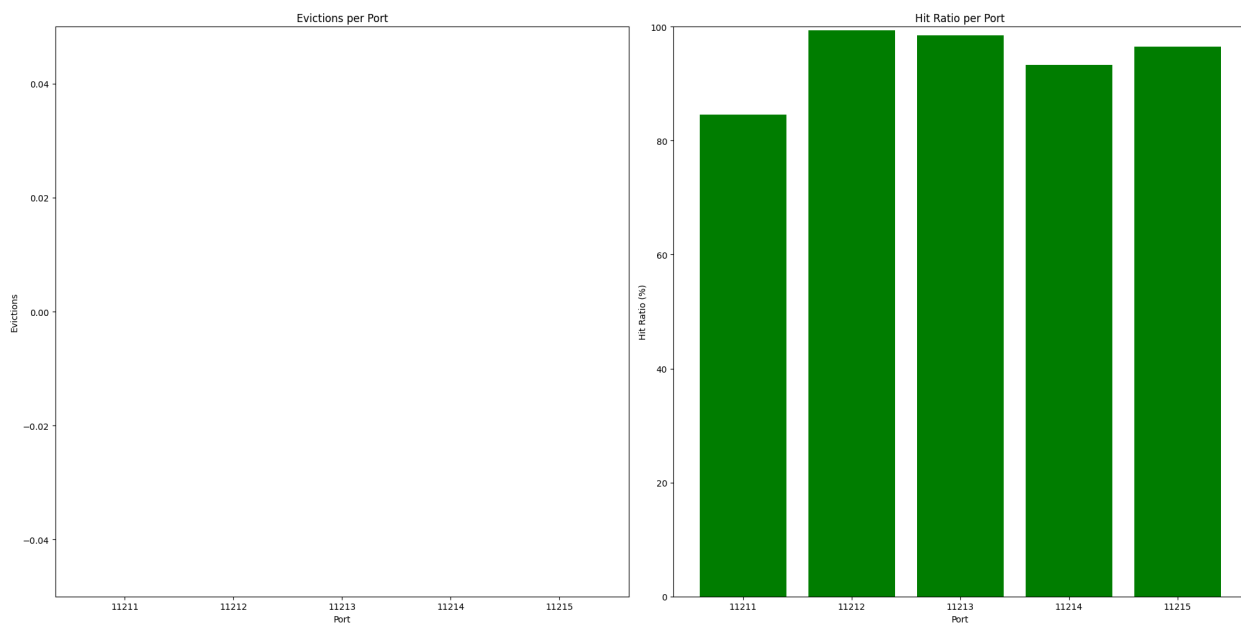
2



3

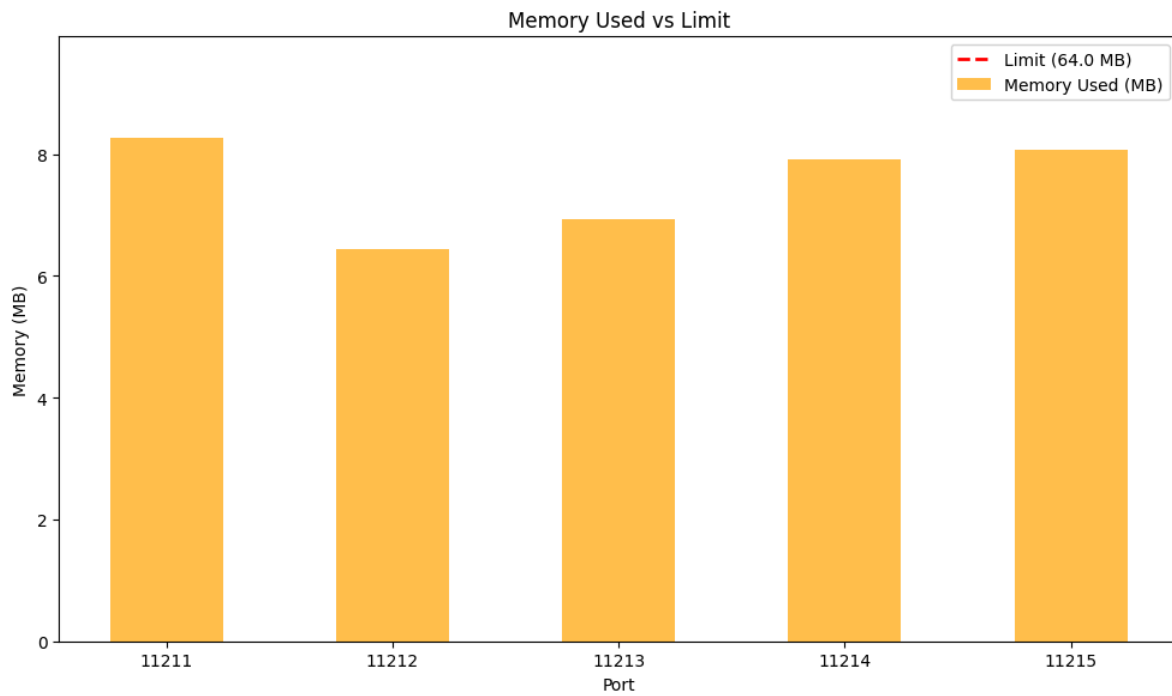


4

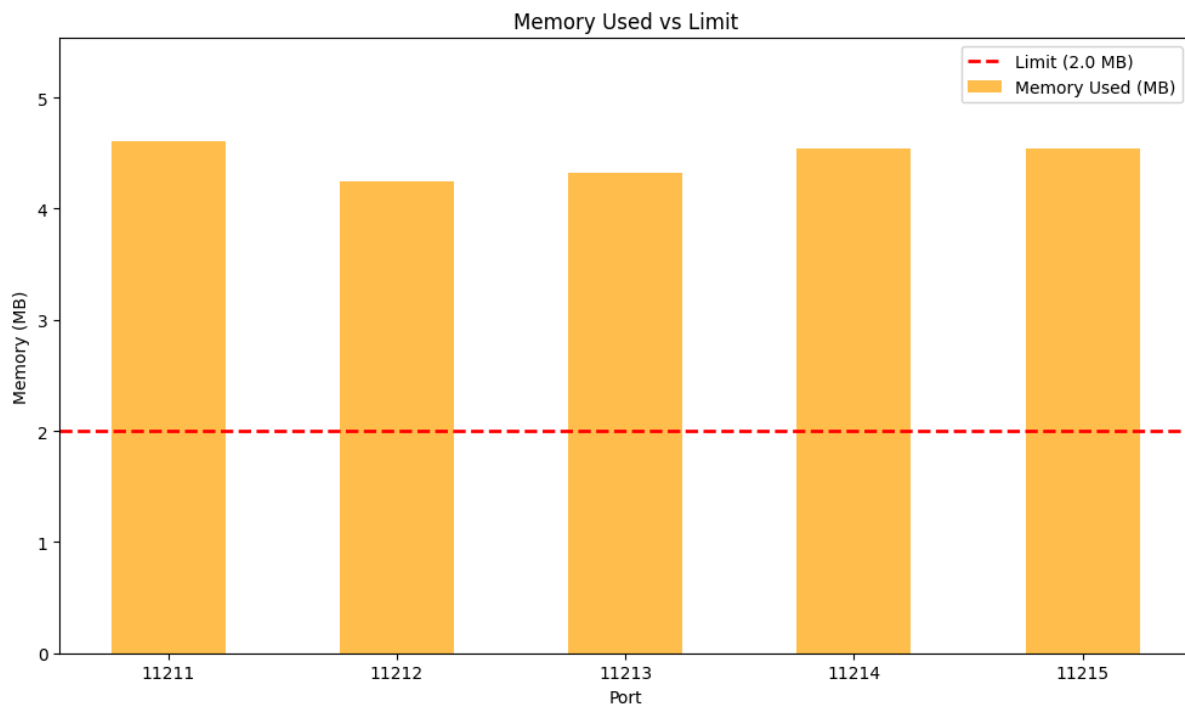


Memory

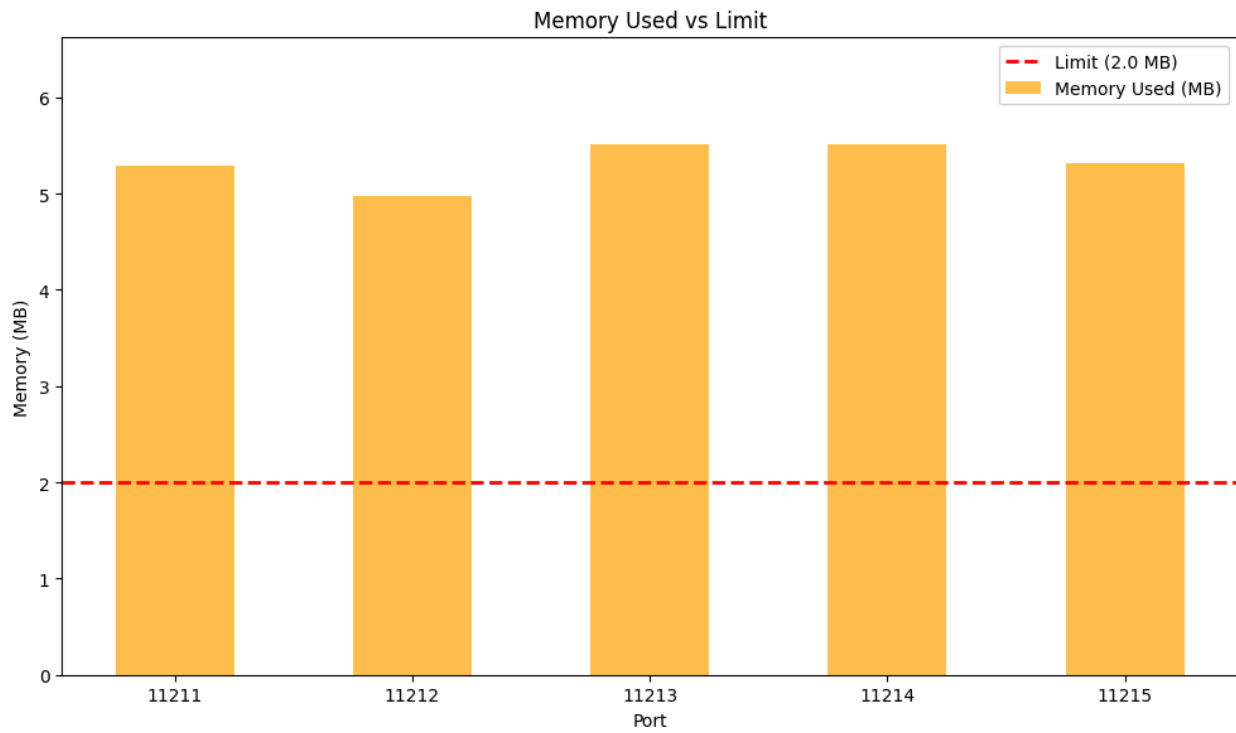
1



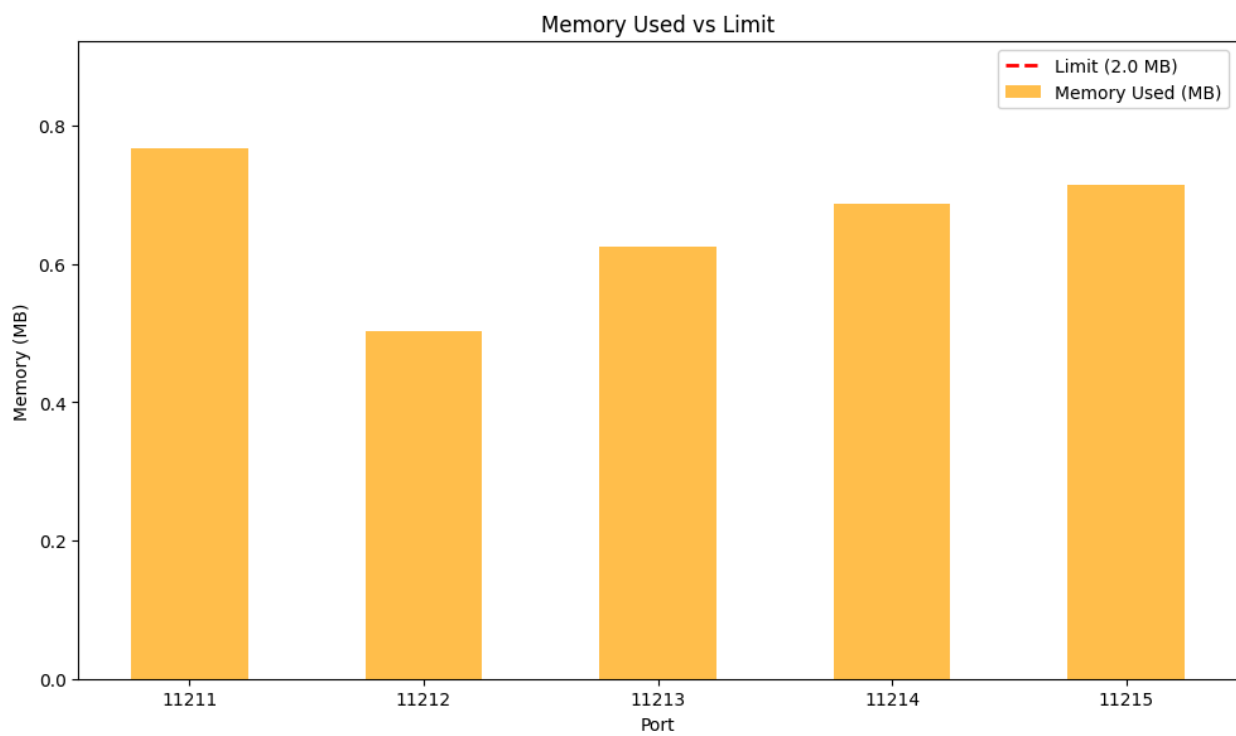
2



3



4



Final print out in the terminal

1

```
isabelledarling@Isabelles-MacBook-Air memcache-8.2 % python3 inject.py
injection: 3.79 seconds
Port: 11211
--Hits: 3934
--Misses: 2825
--Bytes Used: 8680541
--Bytes Written: 143012996
--Items: 245
--Write Attempts: 675
--Read Attempts: 5959
--Evictions: 0

Port: 11212
--Hits: 10033
--Misses: 1874
--Bytes Used: 6751181
--Bytes Written: 362943713
--Items: 192
--Write Attempts: 1277
--Read Attempts: 11907
--Evictions: 0

Port: 11213
--Hits: 8165
--Misses: 1936
--Bytes Used: 7263659
--Bytes Written: 291373878
--Items: 205
--Write Attempts: 1151
--Read Attempts: 10101
--Evictions: 0

Port: 11214
--Hits: 8351
--Misses: 2220
--Bytes Used: 8307057
--Bytes Written: 297619409
--Items: 242
--Write Attempts: 1160
--Read Attempts: 10571
--Evictions: 0

Port: 11215
--Hits: 4584
--Misses: 1879
--Bytes Used: 8456391
--Bytes Written: 159392858
--Items: 239
--Write Attempts: 736
--Read Attempts: 6463
--Evictions: 0
```

```
axs[0].set_ylim(0, max(g_evic
final hit ratios:
Port 11211: 66.02%
Port 11212: 84.26%
Port 11213: 80.83%
Port 11214: 79.00%
Port 11215: 70.93%
```

2

```

injection: 3.97 seconds
Port: 11211
--Hits: 3884
--Misses: 2148
--Bytes Used: 4836111
--Bytes Written: 132457415
--Items: 159
--Write Attempts: 671
--Read Attempts: 6032
--Evictions: 83
-----
Port: 11212
--Hits: 10260
--Misses: 1694
--Bytes Used: 4451442
--Bytes Written: 359515332
--Items: 141
--Write Attempts: 1331
--Read Attempts: 11954
--Evictions: 65
-----
Port: 11213
--Hits: 7994
--Misses: 1968
--Bytes Used: 4527633
--Bytes Written: 267828781
--Items: 152
--Write Attempts: 1088
--Read Attempts: 9962
--Evictions: 49
-----
Port: 11214
--Hits: 8125
--Misses: 2295
--Bytes Used: 4758618
--Bytes Written: 288569847
--Items: 157
--Write Attempts: 1103
--Read Attempts: 10420
--Evictions: 70
-----
Port: 11215
--Hits: 4716
--Misses: 1979
--Bytes Used: 4757617
--Bytes Written: 167028295
--Items: 154
--Write Attempts: 744
--Read Attempts: 6695
--Evictions: 74
-----
final hit ratios:
Port 11211: 64.39%
Port 11212: 85.83%
Port 11213: 80.24%
Port 11214: 77.98%
Port 11215: 70.44%
○ isabelledarling@Isabelles-MacBook-Air memcache-8.2 %

```

```

isabelledarling@Isabelles-MacBook-Air memcache-8.2 % python3 inject.py
injection: 5.65 seconds
Port: 11211
--Hits: 8339
--Misses: 3709
--Bytes Used: 5552487
--Bytes Written: 280918797
--Items: 202
--Write Attempts: 1325
--Read Attempts: 12048
--Evictions: 225
-----
Port: 11212
--Hits: 20791
--Misses: 3070
--Bytes Used: 5210180
--Bytes Written: 731787292
--Items: 190
--Write Attempts: 2666
--Read Attempts: 23861
--Evictions: 175
-----
Port: 11213
--Hits: 16652
--Misses: 3444
--Bytes Used: 5779353
--Bytes Written: 578705073
--Items: 213
--Write Attempts: 2242
--Read Attempts: 20096
--Evictions: 159
-----
Port: 11214
--Hits: 16866
--Misses: 3982
--Bytes Used: 5785253
--Bytes Written: 595573256
--Items: 213
--Write Attempts: 2197
--Read Attempts: 20848
--Evictions: 198
-----
Port: 11215
--Hits: 9712
--Misses: 3514
--Bytes Used: 5568860
--Bytes Written: 340758183
--Items: 195
--Write Attempts: 1491
--Read Attempts: 13226
--Evictions: 224
-----
final hit ratios:
Port 11211: 69.21%
Port 11212: 87.13%
Port 11213: 82.86%
Port 11214: 80.90%
Port 11215: 73.43%
○ isabelledarling@Isabelles-MacBook-Air memcache-8.2 %

```

Ln 54, Col 24 Spaces: 4 U

```

● isabelledarling@Isabelles-MacBook-Air memcache-8.2 % python3 inject.py
injecting data
injection: 2.37 seconds
Port: 11211
--Hits: 1115
--Misses: 204
--Bytes Used: 804946
--Bytes Written: 37520883
--Items: 24
--Write Attempts: 149
--Read Attempts: 1319
--Evictions: 0

-----

Port: 11212
--Hits: 27881
--Misses: 167
--Bytes Used: 527640
--Bytes Written: 987663798
--Items: 15
--Write Attempts: 3134
--Read Attempts: 28048
--Evictions: 0

-----

Port: 11213
--Hits: 9426
--Misses: 145
--Bytes Used: 655411
--Bytes Written: 327773780
--Items: 22
--Write Attempts: 1111
--Read Attempts: 9571
--Evictions: 0

-----

Port: 11214
--Hits: 2033
--Misses: 147
--Bytes Used: 720150
--Bytes Written: 72549339
--Items: 20
--Write Attempts: 238
--Read Attempts: 2180
--Evictions: 0

-----

Port: 11215
--Hits: 3701
--Misses: 134
--Bytes Used: 748818
--Bytes Written: 129976347
--Items: 23
--Write Attempts: 415
--Read Attempts: 3835
--Evictions: 0

-----

/Users/isabelledarling/school/cs4675/memcache-8.2/inject.py:192: UserWarning: Attempting to set identical low and
atically expanding.
  axs[0].set_ylim(0, max(g_evictions) * 1.2)
final hit ratios:
Port 11211: 84.53%
Port 11212: 99.40%
Port 11213: 98.49%
Port 11214: 93.26%
Port 11215: 96.51%
○ isabelledarling@Isabelles-MacBook-Air memcache-8.2 %

```

Explanation of the graphs

As we can see from the graphs above the results are similar despite the differences in parameters (with the exception of 4 which is expected). Notably in run 1 there were no evictions because of the larger memory size. In 2, 3 evictions are seen. The hit ratio for all of the runs was notably similar even with the evictions. The hit ratio varied from around 64% for the first port to around

85% for the 3rd port. These similar results are likely due to the same distribution being used. however the results aren't the exact same and changing the initial parameters will result in slightly different results. This is the same for all graphs. For bytes used and written the used bytes is slightly larger for 1 than compared to 2, 3 this is due to more data being allotted for that run.

Notable Relationships

As the ratio of total users to memory allotted goes up so does the bytes used which is self explanatory. But even as the number of users and data goes up the ratio does not change and the ratios of each static seem relatively the same.

There is a relationship between items in a port and the memory used which is self explanatory. In addition more items in the port also seems to lower the hit ratio which also makes sense because as more items are added the chance of a hit miss goes up due to more space being used in the hash table.

There is somewhat of a negative relationship between evictions and hit ratio. Relatively the larger eviction counts have lower hit ratios but this is not absolute. This makes sense because the number of items can vary and as evictions go up then it causes slow downs and hit rate can go down but this depends on other variables such as the hashing.

The section that requires the most explaining is the memory section. There is a dotted red line across the y axis to represent the max bytes allocated to that port. In the first graph this line is not present as the memory used only goes up to about 8 while the limit is 64 bytes. This explains why no evictions were needed since the memory was not even close to being used up; no evictions were made in the cache. 2 and 3 tell a different story. In both these graphs the memory

used goes above the line for all ports, thus more evictions were made. If we compare this graph with the other graphs we will see a positive relationship between memory used and evictions. We will also see a negative relationship with memory used and hits, bytes written, and read attempts.

Notes on graph 4 and shape parameters

The results for the run 4 are different. Its obvious that one port takes most of the items this is because the shape parameter is set at 2. This run was included to show the importance of the shape parameter and how it works. Another notable change due to the shape parameter is a much higher hit ratio and a lower percentage of memory used. This makes sense due to the nature of a cache. If the most common items are being hit much more often than the second most common than those will be accessed straight from the cache. While if more unique items are being hit the cache will have more misses.

Reflection

Through this homework I learned primarily how peer to peer systems work and more importantly how they speed up and help programs. I learned this through research of the options of the homework. That being said Memcached is not a peer to peer system, rather it is a distributed client server architecture. While peer to peer networks are decentralized and each node is treated as a client and a server, Memcached is centralized. This makes it easier to manage and it is more predictable. On the other hand memcached is a cache. The data is lost and only held temporarily. What is interesting to me is that memcached takes a problem that we have talked about regarding peer to peer systems and solves it in a different way. If we have some system that wants to share data to a large amount of other devices we often can use a peer to peer system to share the data in

pieces and speed up the transfer. Memcached fixes this problem in a different way. The problem is that many devices can overload the network while all asking for the same document.

Memcached creates a buffer and stores the data in cache so it can give out that data quickly so the database doesn't have to do the same thing again and again. This shows me that there are many ways to solve one problem.

I got to work with libraries I am familiar with in python like matplotlib and numpy but I also learned how to use memcached and saw how it is very scalable and easy to use. Even with a large amount of data it still works relatively fast. In addition I was able to create graphs with all the stats it gives which is helpful. Compared to the last homework the system was a lot easier to use and everything was well documented which was nice.

Input and Output Model:

In our case our input is the initial 5 peers that we create. Unlike the previous assignment there isn't a clear output as this tool is used to speed up query retrieval rather than specifically return something. But we can assume the statistics of the ports and their performance is our output. In my case the [inject.py](#) code takes the running nodes, injects a parameterized amount of data into each port and returns output graphs and prints out data in the terminal. The results of all of these inputs and outputs are in earlier sections.